

EXP NO: 8

MODEL EVALUATION AND IMPROVEMENT: HYPERPARAMETER TUNING WITH GRID SEARCH AND CROSS-VALIDATION

AIM

To evaluate and improve a machine learning model using **Grid Search for hyperparameter tuning** and **K-Fold Cross-Validation** to obtain reliable model performance and reduce overfitting.

PROCEDURE

1. Load the Iris dataset.
2. Split the dataset into training and testing sets.
3. Standardize the input features.
4. Define an SVM model with different hyperparameter values.
5. Apply Grid Search with 5-fold cross-validation.
6. Select the best hyperparameters and evaluate the tuned model.
7. Apply K-Fold Cross-Validation and calculate accuracy.
8. Display the confusion matrix and cross-validation scores.

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV, KFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

param_grid = {
```

```

'C': [0.1, 1, 10, 100],
'gamma': [1, 0.1, 0.01, 0.001],
'kernel': ['rbf', 'linear']
}

grid = GridSearchCV(
    SVC(),
    param_grid,
    cv=5,
    scoring='accuracy'
)

grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
print("Best CV Accuracy:", round(grid.best_score_, 4))

model = grid.best_estimator_

y_pred = model.predict(X_test)

print("Test Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)
plt.title("Confusion Matrix - Tuned SVM")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()

kf = KFold(n_splits=5, shuffle=True, random_state=42)

cv_scores = cross_val_score(
    model,
    scaler.fit_transform(X),
    y,
    cv=kf,
    scoring='accuracy'
)

print("K-Fold CV Scores:", np.round(cv_scores, 4))
print("Mean CV Accuracy:", round(np.mean(cv_scores), 4))
print("Standard Deviation:", round(np.std(cv_scores), 4))

plt.figure(figsize=(7, 5))
plt.bar(range(1, 6), cv_scores)

```

```
plt.axhline(
    np.mean(cv_scores),
    linestyle='--',
    label=f"Mean = {np.mean(cv_scores):.4f}"
)
plt.xlabel("Fold Number")
plt.ylabel("Accuracy")
plt.title("5-Fold Cross-Validation Accuracy")
plt.legend()
plt.show()
```

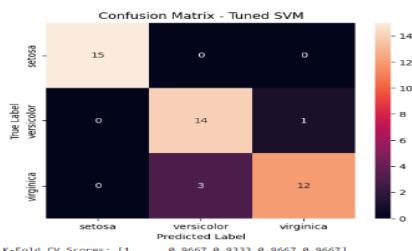
OUTPUT:

```
*** Best Parameters: {'C': 1, 'gamma': 0.1, 'kernel': 'rbf'}
Best CV Accuracy: 0.981
Test Accuracy: 0.9111

Classification Report:
              precision    recall  f1-score   support

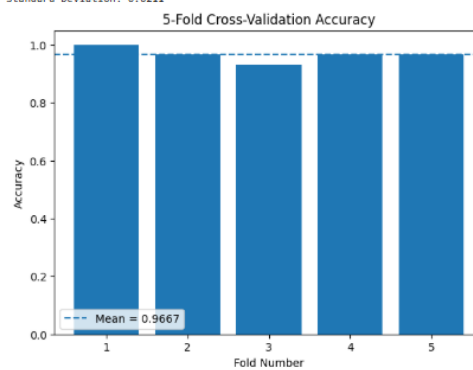
setosa         1.00        1.00        1.00        15
versicolor    0.92        0.93        0.88        15
virginica      0.92        0.80        0.86        15

accuracy         0.92
macro avg        0.92        0.91        0.91        45
weighted avg     0.92        0.91        0.91        45
```



```
K-Fold CV Scores: [1. 0.9667 0.9333 0.9667 0.9667]
Mean CV Accuracy: 0.9667
Standard Deviation: 0.0211
```

```
K-Fold CV Scores: [1. 0.9667 0.9333 0.9667 0.9667]
Mean CV Accuracy: 0.9667
Standard Deviation: 0.0211
```



RESULT:

The SVM model was successfully evaluated and improved using **Grid Search and K-Fold Cross-Validation**. Grid Search identified suitable hyperparameters, while cross-validation provided a reliable estimate of model performance.